

Development Plan

Software Engineering

Team #16, DevSquared
Omar Ghoneim
Meena Al Omran
Marko Popovic
Maya Azar
Amirah Abdelrahman

Table 1: Revision History

Date	Developer(s)	Change
Sep 28th, 2026	All	Revision 0

This document is an overview of the developmental aspects of the [Game 'N' Grow](#) project. This includes specification of team dynamics, project decomposition and scheduling, and intellectual property and copyright specifications. Additionally, technical details such as workflow, expected technology and coding standards, AI/LLM usage, and a Proof of Concept Plan are covered. Overall, the document sets standards and provides a roadmap for the project's development.

1 Confidential Information?

There is no confidential information from industry to protect.

2 IP to Protect

There is no external IP partnership to protect. This project is independently thought of and implemented by the student team.

3 Copyright License

Our team is adopting the MIT license for the sake of simplicity and freedom.

4 Team Meeting Plan

The team will have a weekly in-person meeting; those with extenuating circumstances may join virtually. The default meeting time is Thursday at 12:30pm as none of the team members have lectures.

We plan to meet with our supervisor every 2 weeks on a Zoom call, subject to their availability. Larger development milestones will ideally be showcased in-person to the supervisor.

The meetings will be structured in a manner where the Project Manager comes prepared with an agenda of upcoming deliverables and tasks to be handled. The Project Manager is the meeting chair facilitating the flow of the meeting, such as major updates from each team member. Outstanding work or conflicts are addressed before the end of the meeting. The agenda denotes all decisions made regarding agenda items.

5 Team Communication Plan

The team will use Git Issues with specific labels on a Kanban Board display to track the progress of work items and who is responsible for them. Github branches and pull requests function as a form of version control, history and technical reviews.

Teams will be utilized as the formal method of communication for file sharing, important messages and virtual meetings. An iMessage groupchat serves as an informal means to deliver quick non-sensitive information.

6 Team Member Roles

- Meena Al Omran
 - Administrator: submits deliverables on Avenue2Learn and manages team documents
 - Frontend Developer: responsible for UI/UX design and artistic elements
- Omar Ghoneim
 - Liaison: Point of contact between team and external entities including teaching staff, supervisors, and stakeholders
 - Backend Developer: Responsible for server-side implementation and data management
- Marko Popovic
 - Notetaker: Summarizes meeting minutes and action items during team meetings
 - Game Designer: Responsible for game mechanics and logic, bridging frontend and backend modules
- Maya Azar
 - Meeting Chair: Prepares meeting agendas and directs meeting flow
 - QA and Testing: Implements unit testing and integration testing, oversees overall code quality
- Amirah Abdelrahman
 - Project Manager: Monitors project status and ensures ongoing progress towards deadlines
 - Research and Development: Researches subject matter and consults field experts, ideates technical solutions for stakeholder goals

Note that member roles are subject to change throughout the project; this applies to both technical and administrative roles. Roles for milestone i will be revisited at the first meeting following the completion and submission of milestone $i-1$. This conversation will be initiated by the meeting chair. This entirely depends on the group's preferences so it's possible that, for example, only two members switch their roles and the others retain their current roles. It's also possible that for a given milestone, a group member may take on one technical role for one task and another for a different task, and this will be discussed during that initial meeting as well as throughout the milestone.

7 Workflow Plan

We will be using Git for version control and code review. We will use branches to separate features to ensure modular development. Pull requests will require at least one reviewer to prevent accidental bugs and preserve code quality.

We will be using GitHub Issues assigned to at least one member. Issues will be classified along 3 axes: priority level, bug/feature/refactor and by software module.

Our CI involves pushing code changes to branches frequently and merging sub-branches early to detect integration errors. Our CD involves continually testing code in a staging environment and updating the main branch accordingly.

8 Project Decomposition and Scheduling

We will be using [this GitHub project](#) for project decomposition, scheduling, and progress tracking.

Each major milestone has been defined as a GitHub Milestone with the appropriate title and deadline, and broken down into high-level tasks represented as GitHub Issues linked to the milestones. The list of milestones can be accessed [here](#). As we progress through these milestones, we will identify smaller and more specific tasks of work which we will create new issues for. For milestone i, the assignment of team members to tasks will take place during the first meeting following the completion and submission of milestone i-1. The progression of issues from To Do, to In Progress, to In Review, and finally to Done can be tracked through the [Kanban Board](#).

In addition to the Kanban Board, we have a [Roadmap](#) view in the GitHub project which displays each milestone's due date across a timeline, as well as its associated issues. This is a frame of reference for us to keep track of upcoming work to be completed, as well as our progression. As we continue to create and add more issues for each milestone, the roadmap will continue to be populated.

9 Proof of Concept Demonstration Plan

Risks:

- Different users may find some perspectives not conducive to learning or engaging; for example, being overwhelmed by the nature of 3D perspectives.

What we will demonstrate to convince ourselves we can overcome this risk is designing multiple perspective modes for suitable games (e.g., 2D, first-person 3D, third-person 3D) to alleviate any user issues pertaining to perspective.

If our PoC demo fails regarding this, we may work around this risk by supplanting the 3D ideas with 2.5D rendering instead, replicating games such as Overcooked.

- Users may find some games to be overly stimulating, or not stimulating enough; for example, the noise level in a grocery shopping mini-game.

What we will demonstrate to convince ourselves we can overcome this risk is adding adjustable settings to each game to pinpoint the most suitable level of stimulation for the user.

If our PoC demo fails regarding this, we may simplify the simulation levels to a binary option, or assume low stimulation as the default, since overstimulation is a larger risk.

- Some games may be considered too easy or too difficult depending on the user, missing the goal of learning and progress tracking.

What we will demonstrate to convince ourselves we can overcome this risk is implementing difficulty levels for each game. This enables users to play at a difficulty ideal for their skills and needs.

If our PoC demo fails regarding this, we may implement a binary option regarding difficulties that solely shortens the maximum time given to complete a task within the game, or the game as a whole.

- The caregiver may not have sufficient flexibility or freedom to tailor games to each user; the games not being personable per user.

What we will demonstrate to convince ourselves we can overcome this risk is implementing a template system for the caregiver to create their own games from pre-existing baseline games. For example, the caregiver may create a vocabulary-based mini-game for the user's specific information such as their favorite grocery items.

If our PoC demo fails regarding this, we may create a text-based form per game for the caregiver to input the customized vocabulary, rather than a full template system with pictures, colors et cetera.

- The caregiver may have difficulty interpreting the data collected from the games, and thus be unable to derive useful information for the user.

What we will demonstrate to convince ourselves we can overcome this risk is creating multiple views of the data for the caregiver such as graphical views, table views and summarized bullet point views.

If our PoC demo fails regarding this, we may instead offer a method for the caregiver to export the data to a CSV file, and analyze it themselves in a program such as Excel.

10 Expected Technology

Our team will use a pre-packaged game engine, namely Godot, as the bedrock of our project implementation. Godot is a free and open-source game engine that has a very lenient learning curve via a Pythonic language (GD Script)

made specifically for game development. It is also cross-platform, allowing for enhanced accessibility as the project can run on multiple devices.

AssetLib, Godot's native asset library, enables us to choose assets and add-ons such as: objects & frameworks instead of creating everything from scratch. Godot Unit Testing (GUT) is a unit testing framework that serves as a quick method for creating and executing unit tests.

For frontend purposes, we will largely be using Godot's built-in GUI system, which allows for the creation of user interfaces with minimal code. However, we may use the TauPlot add-on for complex data visualization instead of creating our own math system.

For backend purposes, we are looking at Godotbase as a bridge between our UI layer and potentially our Supabase database layer. Godotbase permits very easy integration with Supabase, while Supabase has strong features for authentication, database management and storage.

SonarQube is a code quality tool that we may use to double as both a linter and a code coverage measuring tool. It is compatible with Godot and can be used to analyze our codebase for potential bugs, vulnerabilities, and code smells.

Our plan for Continuous Integration (CI) is to use GitHub Actions, which is a CI/CD platform that allows us to automate our build, test, and deployment processes. It is integrated with GitHub, making it easy to set up and manage.

11 Use of AI and LLMs

- **Development and Documentation:** LLMs will be used throughout development as a helpful resource for both programming and documentation. They will assist with becoming familiar with specific and game-engine functionality by providing explanations, examples, and implementation guidance.

Verification: The team will focus on testing generated code strictly.

- **Code Generation:** LLMs may be used to generate boilerplate, unit tests, and common code structures to speed up development and avoid wasting time, allowing us to focus on design.

Verification: All generated code will be reviewed by the team to fit with the project's requirements, and go through unit testing and playtesting.

- **Content Bank Generation:** Using AI to generate large banks of game-specific content from a predefined ruleset (such as vocabulary exercises, puzzles, situational problems, etc.) can greatly help with a game's educational value, by encouraging players to learn the skill rather than memorize sets and answers.

Verification: Outputs will be manually reviewed and tested to ensure that they make sense, and align with the specific learning objectives of the given game.

- **Asset Generation:** For audio/visual development, non LLM based tools like stable diffusion and 3D mesh-generation may be used to produce specific sprites, UI assets, or mesh assets if needed for devopment speed.
- Verification:** The generated assets will have to follow a given visual style guide, for clarity and accessibility.

12 Coding Standard

The official GDScript style guide will be followed for all code written in GDScript. This includes things such as naming conventions, code element ordering, and other best practices. The team will also follow industry standard coding habits such as writing clear and concise comments, using meaningful variable and function names, and adhering to the principles of clean code.

The official style guide can be found at https://docs.godotengine.org/en/stable/getting_started/scripting/gdscript/gdscript_styleguide.html.

13 Appendix — Generative AI Use Disclosure

13.1 AI Tools Used

Claude’s Sonnet 5 model.

13.2 How and Where Used

Sonnet was used to learn some of the quirks of LaTeX and fix errors associated with it. This includes the bullet point structure in the Team Member Roles section and paragraphs in the PoC Demo Plan.

13.3 Verification of AI-Generated Content

AI-Generated content was not used in this document. The usage was limited to learning about LaTeX and fixing associated errors.

Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. Why is it important to create a development plan prior to starting the project?

A development plan functions as great documentation to reference when unsure about things, such as our reasoning behind technology choices, our coding standard or the conflict resolution process. Additionally, it enforces teams to think about how project-level tasks will be decomposed amongst the group members to keep progress timely. For example, a dedicated meeting chair avoids confusion regarding who will lead a meeting, or a liaison is the formal point of contact with external entities instead of scattered communication.

2. In your opinion, what are the advantages and disadvantages of using CI/CD?

Advantages of CI/CD include: early bug detections since integration is constantly being tested, higher code quality because merged code must be of a similar strong standard, and lower deployment risk because continuous deployment ensures a functioning source of truth in the main deployment branch. Disadvantages of CI/CD include: a possibly steep learning curve for developers to learn new tools and workflows to actually maintain the pipeline, a high setup price since there is some initial investment setting up the CI/CD pipelines which in our case is setting up a Kanban board and issue labels and such.

3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Our group was largely cohesive this deliverable and only had minor disagreements regarding whether to use Godot or another game engine such as Unity or Unreal. That conflict resolved by discussing the pros and cons of each tool. In the end, the person with Unreal experience conceded that it is a very heavyweight application with a much more difficult learning curve, and Godot is therefore the more suitable choice for our project.

Appendix — Team Charter

External Goals

Our team has an aligned set of goals for this project. We want to create a project that has a real positive impact on people's lives, especially a marginalized group such as autistic individuals, because our aim is to use our acquired skills to leave the world a better place than we found it.

Secondary priorities include winning a department podium award (1st, 2nd, or 3rd place) at the Capstone EXPO, and showcasing a complex project that we can be proud of to potential employers.

Attendance

Expectations

Punctuality and attendance is expected and required for all team members. If a team member is unable to attend a meeting, they must notify the team at least 30 minutes in advance. If a team member is late to a meeting, they must notify the team as soon as possible. If a team member needs to leave a meeting early, they must notify the team prior to the meeting if possible. If a team member is unable to attend multiple meetings in a row, they must provide a valid reason for their absence.

Acceptable Excuse

For deadlines, only critical emergencies can function as an excuse for missing them. This is because the team member should notify other team members as soon as they think they might not be able to meet a deadline, so the team has time to find a solution and distribute the outwork.

For one-time meeting absences, a team member reserves the right to keep their reason for absence private. However, a repeated absence requires a reason for absence and must constitute a reasonable excuse by the rest of the team that deterred them from attending.

In Case of Emergency

Team members are expected to notify the team as soon as possible if they have an emergency so the team can redistribute their work accordingly. If appropriate, the team member should brief the team on the current status of their work, allowing others to pick up where they left off more smoothly.

Accountability and Teamwork

Quality

Team members should be prepared to discuss their work and progress at each meeting. The meeting chair's agenda along with the Kanban board will reflect

the current status of deliverables and work items; each team member is expected to provide updates regarding issued assigned to them.

The quality of deliverables should be strong with a focus on efficiency and clean code. Technical debt is natural and expected, but must be explicitly documented and addressed in a timely manner. Shortcuts and workarounds are only acceptable if they do not degrade the quality of the codebase, or if they are temporary solutions that will be addressed in the near future.

Attitude

We expect all team members to exhibit openness to others' ideas no matter what their own ideas, in the name of respect and collaboration. Cooperation and positive attitudes are an expectation, and any team member who acts otherwise will be addressed by the team to ensure everyone feels comfortable.

For conflicts, it is expected to be resolved between the concerned parties. Polite communication is imperative. If a conflict still remains unresolved, the team members may utilize one or more other team members as mediators to aid in the conflict resolution. The team may then elect to take a vote or bring the issue to the TA or instructor.

Stay on Track

The team has a meeting chair and project manager that together manage the team's progress; they will have the ability to tell if the team is off track. For team members, it is expected that they will complete their work in a timely manner and at a high level. For team members who are struggling to complete their work, the environment should be supportive enough to allow them to ask for help and guidance from other team members. Team members not contributing their fair share will be addressed by the team in a kind manner and encouraged to contribute proportionately if they are able to. If the problem persists, the team may elect to bring the issue to instructor.

Meeting attendance and amount of lines of code committed to the repository will be monitored by team members as a metric for contribution. The consequences of not doing proportionate work relative to others will be addressed internally, with empathy, before it becomes a persistent issue. However, a pattern of not contributing will be brought to the attention of the TA and/or instructor. Team members performing particularly well may be rewarded with praise and retain the right to choose what tasks they prefer to do in the following sprint or deliverable.

Team Building

Our team is already well-acquainted with each other, so we already spend fun time together outside of the project and have a good rapport with respect and trust. We will continue to build team cohesion with activities such as eating meals together, playing games, and celebrating milestones.

Decision Making

Decisions are made after each group member has spoken their opinion. It is important that each member feel valued and genuinely heard, not figuratively. Options will be discussed until a consensus is reached. Ideally, decisions are unanimous, but quick decisions may have to be taken with a majority vote.